

Computer Vision for dummies

Object Detection with YOLO and Roboflow

Chi sono



Simone Bonfrate

XR Engineer @ Wideverse

AI & Data Science Student @ Politecnico di Bari

Community Lead @ GDG Bari

Runner, D&D Player, Manga Reader...

Identifichiamo un problema

Immaginiamo che il sig. Mario Rossi deve andare a raccogliere i funghi.

Mario non sa quali funghi siano velenosi o no.

Cosa può fare Mario per capire quali prendere o lasciare?



Computer Vision

Rappresenta una tecnica per estrapolare informazioni da immagini e video.

Sfrutta tecniche di intelligenza artificiale per estrapolare elementi utili per la comprensione



Computer Vision: Casi d'uso

Google Lens

Tesla

Impianti industriali

Computer Vision: Casi d'uso

Google Lens

Tesla

Impianti industriali



Google Lens

Labradoodle
Dog breed



A Labradoodle, also known as a Labrapoodle, is created by crossing a Labrador Retriever and a Poodle.



Search



Copy



Share

Computer Vision: Casi d'uso

Google Lens

Tesla

Impianti industriali



Computer Vision: Casi d'uso

Google Lens

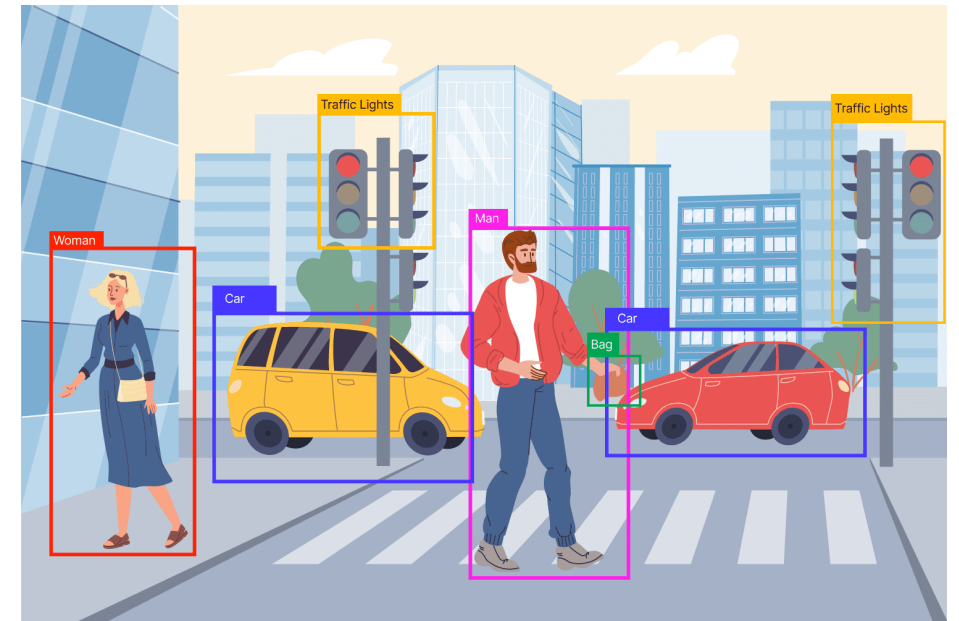
Tesla

Impianti industriali



Object Detection: che cos'è?

E' una tecnica di computer vision che, dato in input un'immagine, permette di distinguere la presenza o meno di oggetti di una determinata classe.



YOLO (You Only Look Once)

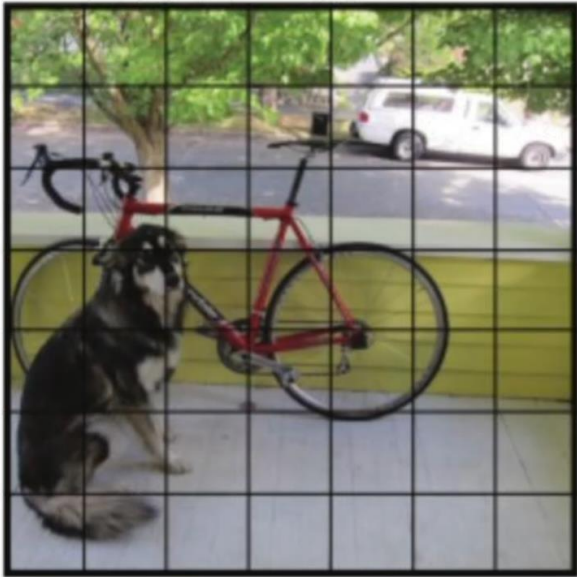
YOLO è un modello di Object detection capace di predire dall'intera immagine una sola volta la posizione e la classe degli oggetti.

Questo approccio consente a YOLO di ottenere risultati in tempo reale. Utile per applicazioni che richiedono una rapida elaborazione.



YOLO (You Only Look Once)

YOLO divide l'immagine in una griglia di celle e per ciascuna ricava sia le bounding box che la classe .



$S \times S$ grid of cells



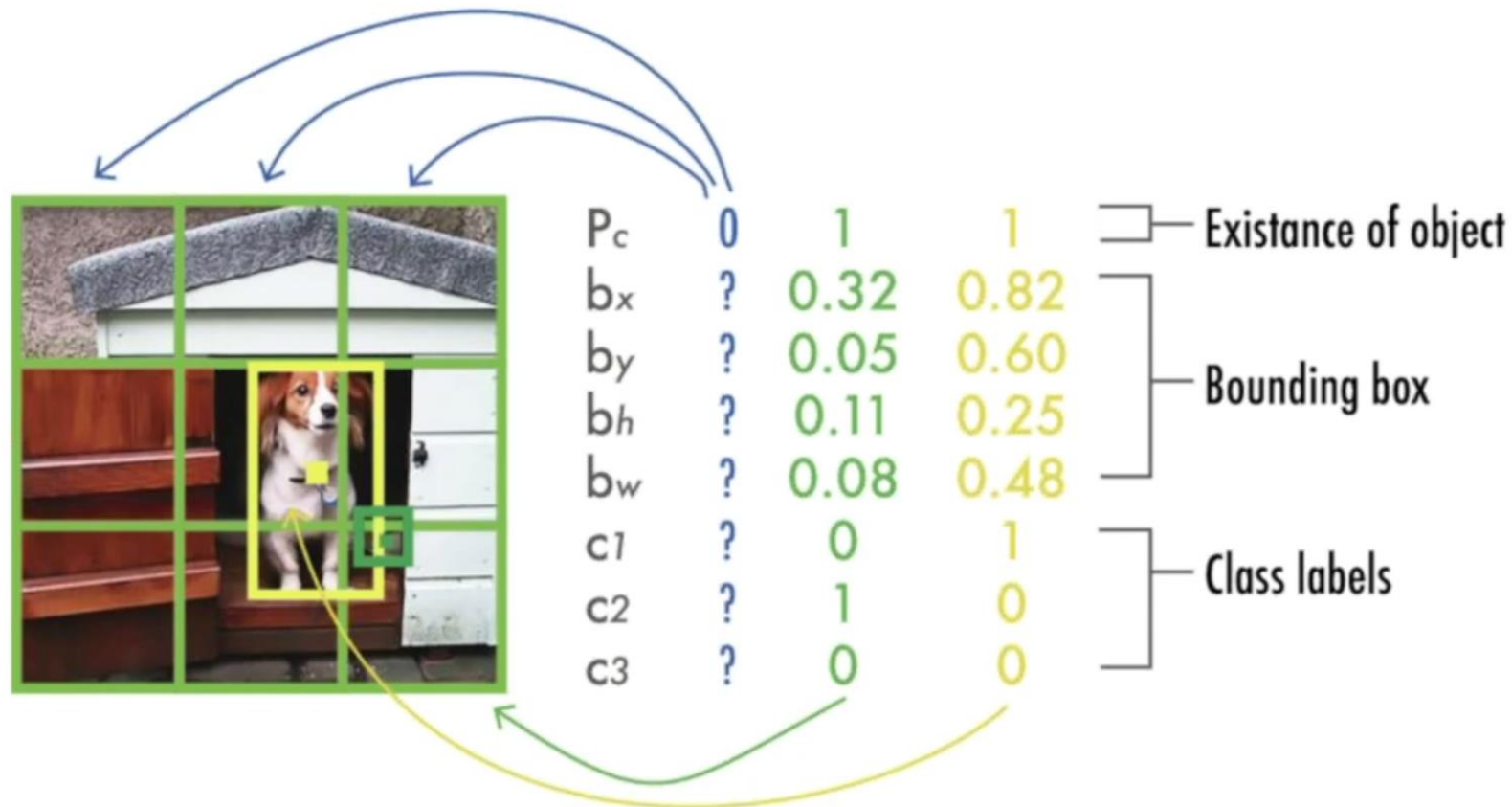
Bounding boxes and confidence



Class probability map

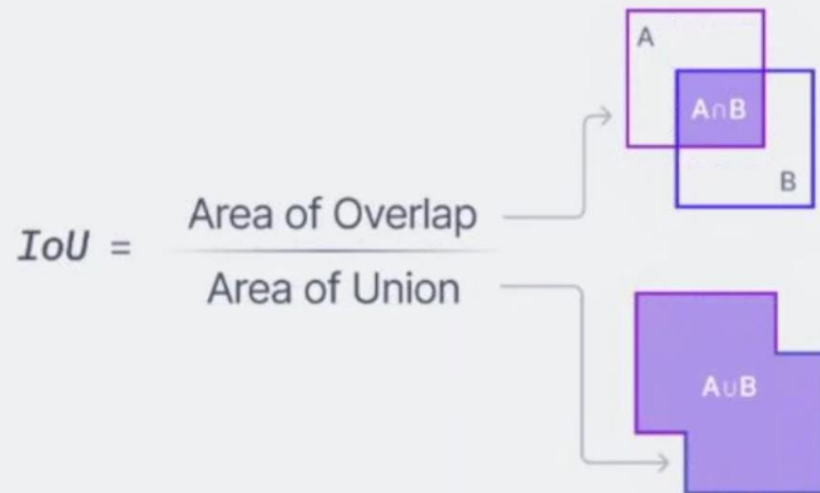
YOLO (You Only Look Once)

L'unione delle classi predette e delle bounding box genera un output vettoriale nel seguente formato



YOLO (You Only Look Once)

Per stimare le performance si usa l'Intersection over Union



$$\text{Intersection over Union } (IoU) = \frac{|A \cap B|}{|A \cup B|}$$

$IoU = 0.35$



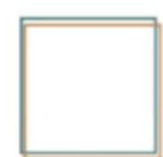
Poor

$IoU = 0.74$



Good

$IoU = 0.93$



Excellent

Preoccupiamoci dei dati

I dati sono l'elemento più importante per qualsiasi applicazione d'intelligenza artificiale.

Preoccupiamoci dei dati

I dati sono l'elemento più importante per qualsiasi applicazione d'intelligenza artificiale.

Perché?

Preoccupiamoci dei dati

I dati sono l'elemento più importante per qualsiasi applicazione d'intelligenza artificiale.

Perché?

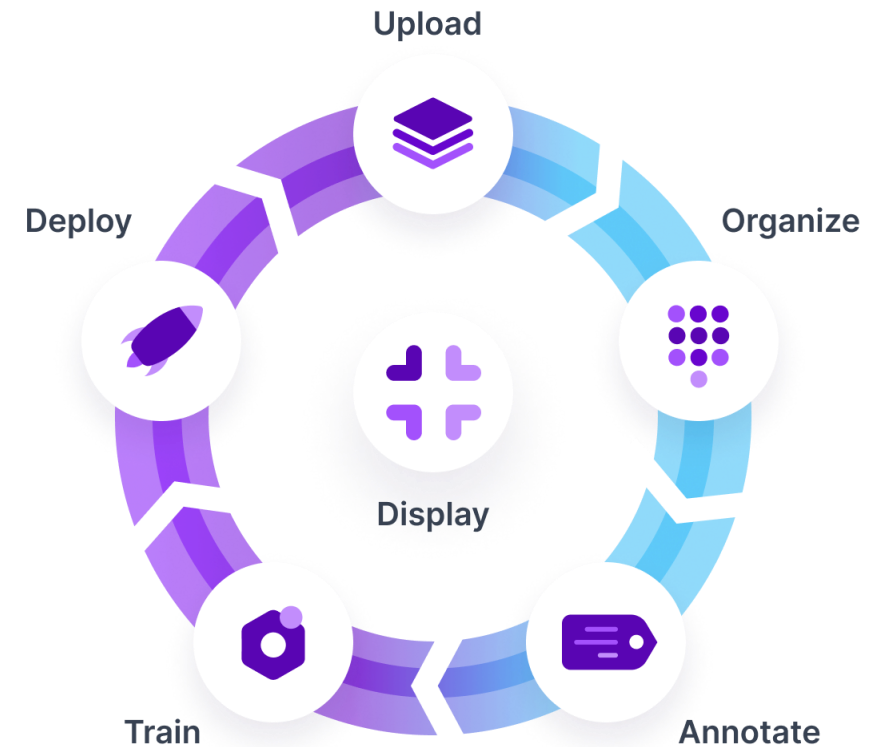
Perché andranno ad impattare sulla qualità dell'addestramento del nostro modello e sulle performance finali.

Garbage In, Garbage Out

Roboflow

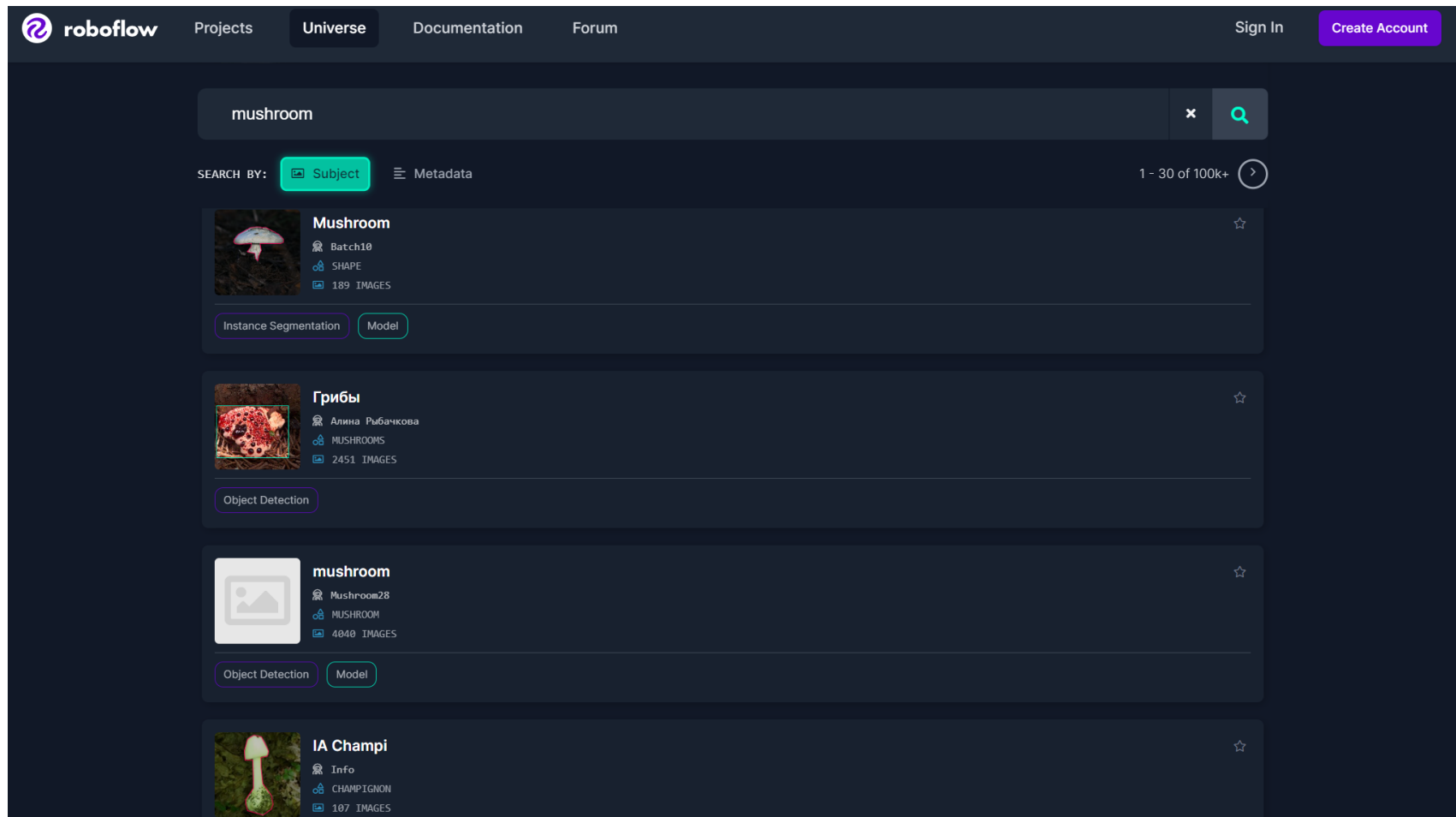
Roboflow è un SaaS per lo sviluppo di applicazioni di Computer vision.

Dispone di servizi dedicati alla realizzazione di dataset, training ed inferenza e deploy



Roboflow Universe

E' un servizio che permette di accedere ad un'infinità di dataset creati dalla community



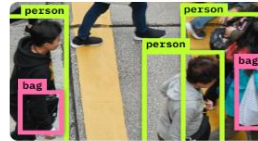
404? Nessun Problema

Possiamo creare un dataset pubblico dove inserire le immagini e le annotazioni.

Create New Project ×

Testing / [New Public Project](#)

Project Type



Object Detection
Find multiple things and their specific location.



Classification
Assign labels to the entire image.



Instance Segmentation
Detect multiple objects and their actual shape.

[Show More](#) ↓

Project Name Cannot be empty.

What are you detecting? ⓘ

License

[Cancel](#) [Create Public Project](#)

Labelling

Prima di dare in pasto le nostre immagini al nostro modello è necessario etichettarle

Manualmente

Roboflow, CVAT

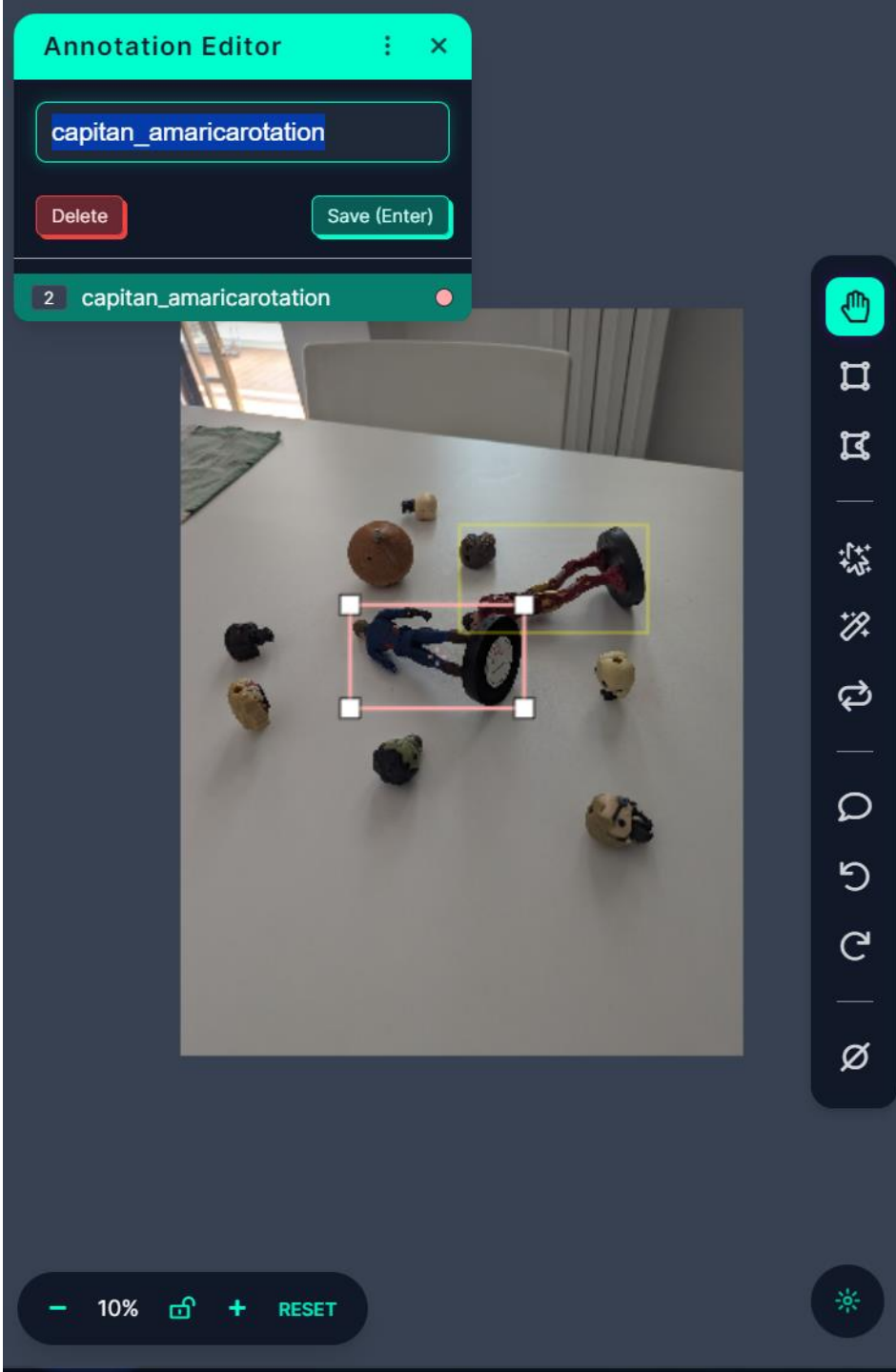
Automaticamente

LangSam, Autodistill

Roboflow Editor

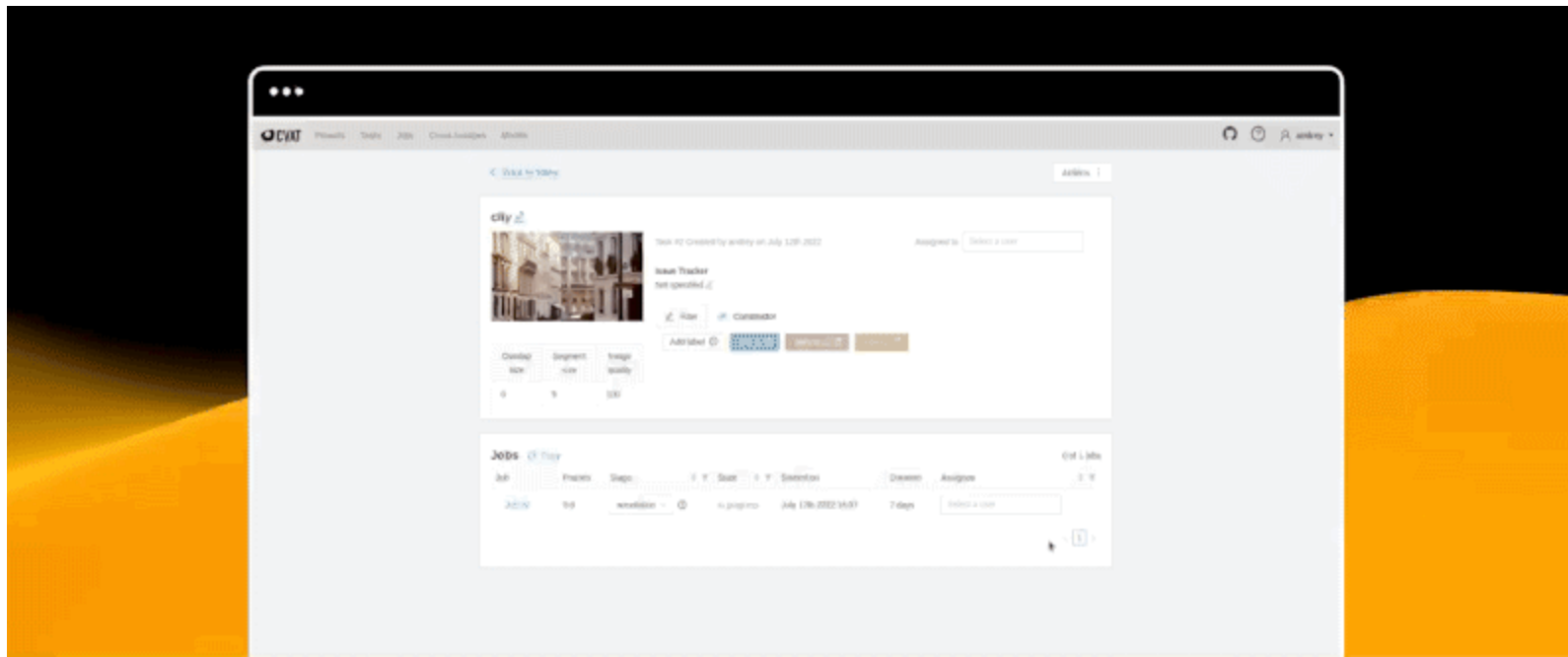
Puoi inserire e modificare le bounding box e definire la classe di riferimento.

Se gli oggetti da rintracciare sono presenti nei dataset più famosi, è possibile usare il Label Assist per generare le bounding box da labellare.



CVAT

E' un'applicazione web open source che fornisce strumenti per il labelling delle immagini.



LangSAM

E' un tool che combina
tecniche di text
prompting
(GroundingDINO) e
segmentazione per la
generazione di maschere
e bounding box

```
from lang_sam import LangSAM
from tqdm import tqdm
from PIL import Image

model = LangSAM()

images = [ ... ]

prompt_label_map = {
    "black screwdriver": "screwdriver",
    "orange plier": "pliers"
}

images_annotations = []
prompt = '.'.join(prompt_label_map.keys())

image_counter = 0

for img in images:
    filename = img.split('/')[-1]
    file_annotation = { "filename": filename, "annotations": [] }

    image_pil = Image.open(img).convert("RGB")

    z_order = 1

    masks, boxes, phrases, logits = model.predict(
        image_pil,
        prompt,
        text_threshold=.5,
        box_threshold=.6)

    labels = [prompt_label_map[phrase] for phrase in phrases]

    for label, box, logit in zip(labels, boxes, logits):
        x_min, y_min, x_max, y_max = box

        x1 = x_min.item()
        x2 = x_max.item()
        y1 = y_min.item()
        y2 = y_max.item()
        conf = logit.item()

        prompt_annotation = {
            "label": label,
            "box": { "x_min" : x1, "x_max" : x2, "y_min" : y1, "y_max" : y2},
            "confidence": conf
        }

        file_annotation["annotations"].append(prompt_annotation)
```

autodistill

Libreria che permette di creare modelli allenati partendo da immagini non etichettate tramite prompt engineering.

```
from autodistill_grounded_sam import GroundedSAM
from autodistill.detection import CaptionOntology
from autodistill_yolov8 import YOLOv8
```

```
base_model = GroundedSAM(
    ontology=CaptionOntology(
        {"shipping container": "container"}
    )
)
```

```
base_model.label(
    input_folder="./images",
    output_folder="./dataset"
)
```

```
target_model = YOLOv8("yolov8n.pt")
target_model.train(
    "./dataset/data.yaml",
    epochs=200
)
```

```
# run inference on the new model
pred = target_model.predict(
    "./dataset/valid/your-image.jpg",
    confidence=0.5
)
```

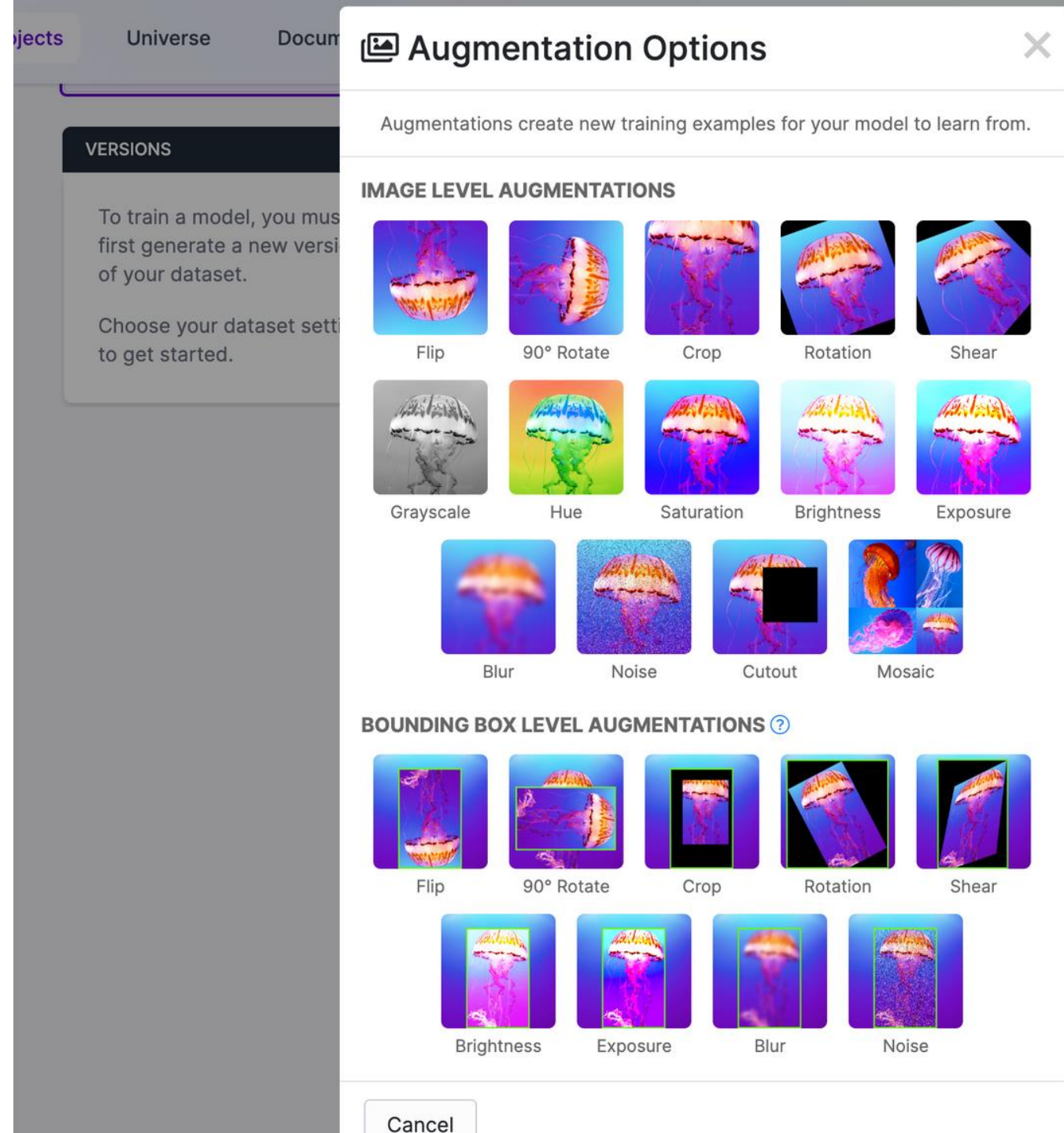
```
print(pred)
```

Augmentation

Spesso i dati che si hanno a disposizione non sono mai sufficienti.

L'augmentation crea più versioni della stessa immagine con alcune modifiche.

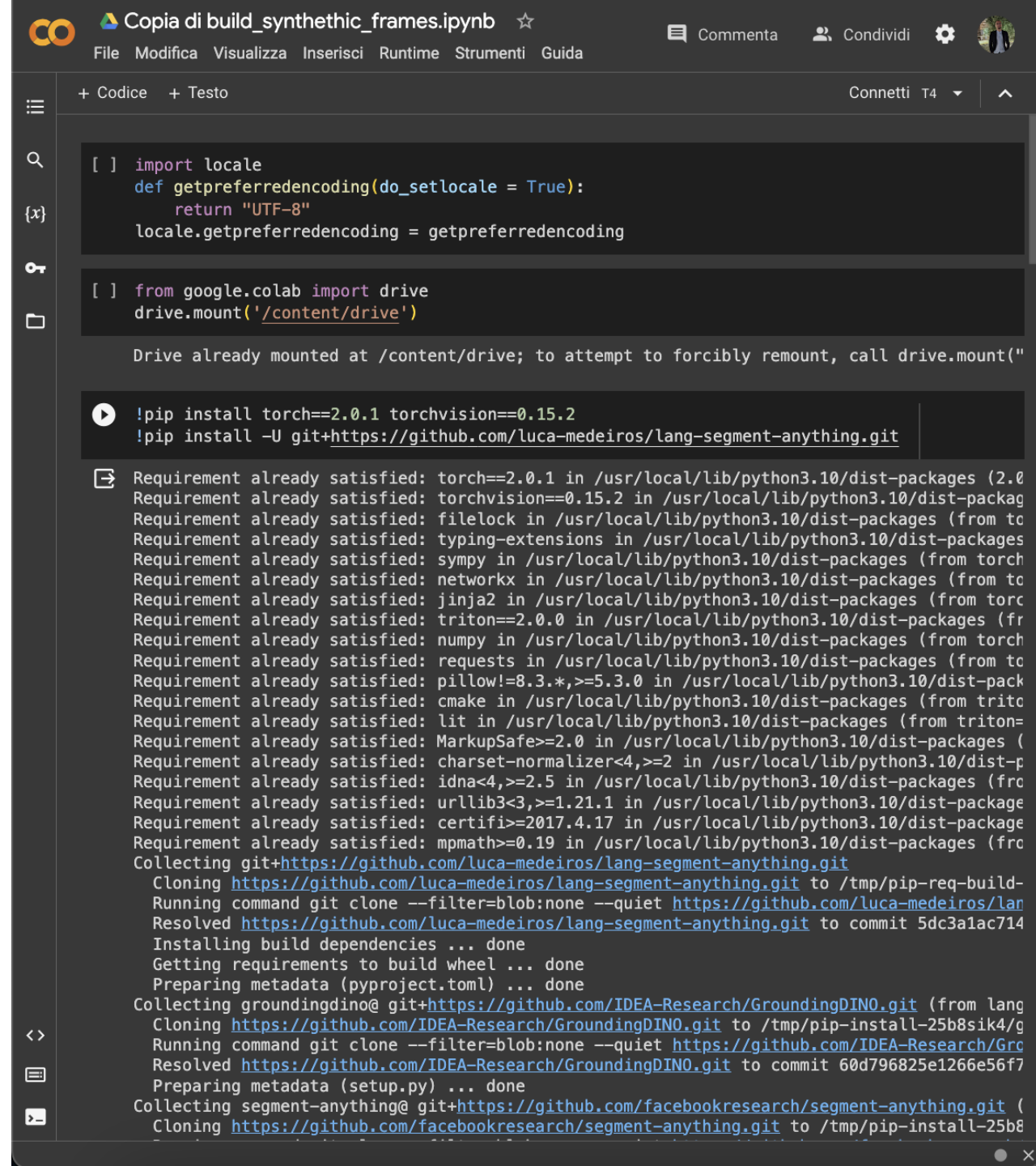
Anche se l'immagini create condividono lo stesso oggetto, consente di irrobustire il modello rendendolo capace di generalizzare.



Prepariamo l'ambiente per il training

Per effettuare il training di YOLO è necessaria molta potenza grafica.

Tramite Google Colab è possibile accedere ad ambienti Python pronti per lo sviluppo dell'IA gratuitamente



The screenshot shows a Google Colab notebook titled "Copia di build_synthetic_frames.ipynb". The interface includes a top bar with navigation icons and a menu (File, Modifica, Visualizza, Inserisci, Runtime, Strumenti, Guida). The notebook has two tabs: "Codice" (active) and "Testo". The code cell contains the following Python code:

```
[ ] import locale
def getpreferredencoding(do_setlocale = True):
    return "UTF-8"
locale.getpreferredencoding = getpreferredencoding

[ ] from google.colab import drive
drive.mount('/content/drive')
```

Below the code, a message states: "Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("".

The output section shows the execution of pip install commands and a list of requirements already satisfied:

```
!pip install torch==2.0.1 torchvision==0.15.2
!pip install -U git+https://github.com/luca-medeiros/lang-segment-anything.git

Requirement already satisfied: torch==2.0.1 in /usr/local/lib/python3.10/dist-packages (2.0
Requirement already satisfied: torchvision==0.15.2 in /usr/local/lib/python3.10/dist-packag
Requirement already satisfied: filelock in /usr/local/lib/python3.10/dist-packages (from to
Requirement already satisfied: typing-extensions in /usr/local/lib/python3.10/dist-packages
Requirement already satisfied: sympy in /usr/local/lib/python3.10/dist-packages (from torch
Requirement already satisfied: networkx in /usr/local/lib/python3.10/dist-packages (from to
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.10/dist-packages (from torc
Requirement already satisfied: triton==2.0.0 in /usr/local/lib/python3.10/dist-packages (fr
Requirement already satisfied: numpy in /usr/local/lib/python3.10/dist-packages (from torch
Requirement already satisfied: requests in /usr/local/lib/python3.10/dist-packages (from to
Requirement already satisfied: pillow!=8.3.*,>=5.3.0 in /usr/local/lib/python3.10/dist-pack
Requirement already satisfied: cmake in /usr/local/lib/python3.10/dist-packages (from trito
Requirement already satisfied: lit in /usr/local/lib/python3.10/dist-packages (from triton=
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.10/dist-packages (
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.10/dist-p
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-packages (fro
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.10/dist-package
Requirement already satisfied: mpmath>=0.19 in /usr/local/lib/python3.10/dist-packages (fro
Collecting git+https://github.com/luca-medeiros/lang-segment-anything.git
  Cloning https://github.com/luca-medeiros/lang-segment-anything.git to /tmp/pip-req-build-
  Running command git clone --filter=blob:none --quiet https://github.com/luca-medeiros/lan
  Resolved https://github.com/luca-medeiros/lang-segment-anything.git to commit 5dc3a1ac714
  Installing build dependencies ... done
  Getting requirements to build wheel ... done
  Preparing metadata (pyproject.toml) ... done
Collecting groundingdino@ git+https://github.com/IDEA-Research/GroundingDINO.git (from lang
  Cloning https://github.com/IDEA-Research/GroundingDINO.git to /tmp/pip-install-25b8sik4/g
  Running command git clone --filter=blob:none --quiet https://github.com/IDEA-Research/Grc
  Resolved https://github.com/IDEA-Research/GroundingDINO.git to commit 60d796825e1266e56f7
  Preparing metadata (setup.py) ... done
Collecting segment-anything@ git+https://github.com/facebookresearch/segment-anything.git (
  Cloning https://github.com/facebookresearch/segment-anything.git to /tmp/pip-install-25b8
```

Installiamo le dipendenze

```
!pip install torch==2.0.1 torchvision==0.15.2
```

```
!git clone https://github.com/ultralytics/yolov5 # clone repo
```

```
%cd yolov5
```

Scarichiamo il dataset

```
!pip install -q roboflow
```

```
%cd /content/yolov5
```

```
from roboflow import Roboflow  
rf = Roboflow(api_key="<API_KEY>")  
project = rf.workspace("<WORKSPACE>").project("<PROJECT>")  
dataset = project.version(1).download("yolov5")
```

Configuriamo il dataset per YOLO

```
# define number of classes based on YAML
import yaml

with open(dataset.location + "/data.yaml", 'r') as stream:
    num_classes = str(yaml.safe_load(stream)['nc'])

#this is the model configuration we will use for our tutorial
%cat /content/yolov5/models/yolov5s.yaml

from IPython.core.magic import register_line_cell_magic

@register_line_cell_magic
def writetemplate(line, cell):
    with open(line, 'w') as f:
        f.write(cell.format(**globals())))
```

Configuriamo il dataset per YOLO

```
%%writetemplate /content/yolov5/models/custom_yolov5s.yaml
```

```
# parameters
```

```
nc: {num_classes} # number of classes
```

```
depth_multiple: 0.33 # model depth multiple
```

```
width_multiple: 0.50 # layer channel multiple
```

```
# anchors
```

```
anchors:
```

```
...
```

```
# YOLOv5 backbone
```

```
backbone:
```

```
...
```

```
# YOLOv5 head
```

```
head:
```

```
...
```

Alleniamo il modello

```
# train yolov5s on custom data for 100 epochs
# time its performance
%%time
%cd /content/yolov5/
!python train.py --img 416
                    --batch 16
                    --epochs 20
                    --data {dataset.location}/data.yaml
                    --cfg ./models/custom_yolov5s.yaml
                    --weights 'yolov5s.pt'
                    --name yolov5s_results
                    --cache
```

Testiamo il modello

when we ran this, we saw .007 second inference time. That is 140 FPS on a TESLA P100!

use the best weights!

```
%cd /content/yolov5/
```

```
!python detect.py
```

```
--weights runs/train/yolov5s_results/weights/best.pt
```

```
--img 416
```

```
--conf 0.4
```

```
--source {dataset.location}/test/images
```


Esportiamo il modello

```
%cd /content/yolov5/  
!python export.py  
    --weights runs/train/yolov5s_results/weights/best.pt  
    --include torchscript  
    --img 416  
    --optimize  
    --conf-thres 0.7
```

Demo

Domande?

Grazie per l'attenzione



bonfry

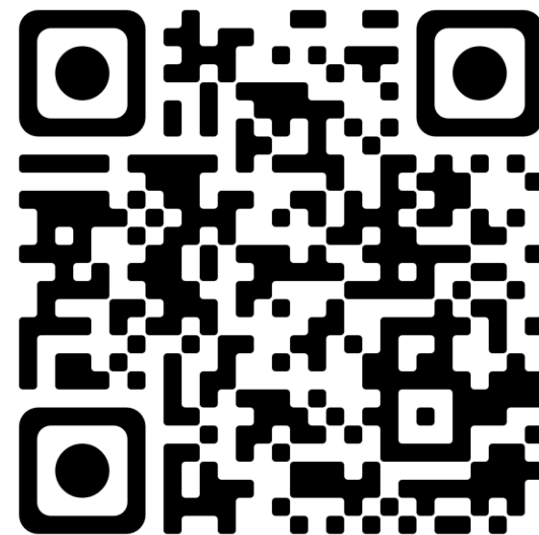


Simone Bonfrate

Riferimenti

Roboflow Youtube channel

Ultralytics LCC



Lascia un feedback